

Homework 3

Comparing Classification Algorithms with Feature Selection

Learning Objectives

This assignment goes beyond simply running classifiers. The goal is to build the skill of comparing several models fairly and interpreting the results. By the end of this assignment, you should be able to:

- Load and preprocess a real UCI dataset into a form suitable for analysis.
- Use five scikit-learn classifiers in an object-oriented fashion.
- Compare model performance statistically using RepeatedKfold cross validation.
- Empirically investigate how feature selection affects each algorithm.
- Document and share your experiments as a GitHub README report.

Overview

In this assignment, you apply the AI fundamentals introduced in the second half of the semester. You will run several classification algorithms on a real dataset and compare their performance. You will also experiment with feature selection to find the most informative subset of input variables.

The assignment has three parts. In Part 1 you choose and preprocess the dataset. In Part 2 you compare five classifiers under the same evaluation protocol. In Part 3 you investigate whether selecting a subset of features can further improve each algorithm.

Recommended Environment

- Python `>= 3.10`
- `scikit-learn >= 1.3`, `pandas`, `numpy`, `matplotlib`
- A Jupyter Notebook (.ipynb) is recommended so that tables and plots can live alongside the code.
- Use a fixed seed (e.g., `random_state=17342`) in every experiment to keep results reproducible.

Part 1. Dataset Selection and Preprocessing

1-1. Choose a Dataset

Each student must choose one classification dataset from the UCI Machine Learning Repository.

To avoid duplicates, register your name and chosen dataset in the shared Google Sheet below. Datasets are assigned on a first-come, first-served basis. Once a dataset is taken, it cannot be selected by another student.

- **Registration link:** [Insert Google Sheet link]

Make sure your dataset is suitable for classification (i.e., the target variable is categorical). Choosing a regression dataset will make it difficult to complete this assignment.

Dataset Selection Checklist

- Is the target variable a clear set of class labels (e.g., positive/negative, type A/B/C)?
- Are there enough samples? At least 200 are recommended; around 1,000 is convenient to work with.
- Is the number of features manageable for Part 3? $m \leq 30$ is recommended.
- Is the class distribution reasonably balanced?

1-2. Preprocessing

Apply the following preprocessing steps to your chosen dataset:

- **Missing values:** drop rows that contain any missing value.
- **Drop irrelevant columns:** remove columns that do not help with classification (IDs, indices, unrelated metadata).
- **Encoding:** encode categorical features using LabelEncoder or one-hot encoding.
- **Scaling:** standardize numerical features (e.g., StandardScaler). This is especially important for KNN and MLP.

Reference Code: Preprocessing Pipeline

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, LabelEncoder

# 1) Load the dataset (e.g., UCI Wine)
df = pd.read_csv('your_dataset.csv')

# 2) Drop rows containing any missing values
df = df.dropna()

# 3) Drop irrelevant columns (e.g., ID)
```

```
df = df.drop(columns=['id'], errors='ignore')

# 4) Separate target and features
y = df['target']
X = df.drop(columns=['target'])

# 5) Encode categorical features
for col in X.select_dtypes(include='object').columns:
    X[col] = LabelEncoder().fit_transform(X[col])

# 6) Encode the target if it is a string
if y.dtype == 'object':
    y = LabelEncoder().fit_transform(y)

# 7) Scale numerical features (important for KNN and MLP)
X_scaled = StandardScaler().fit_transform(X)

print('Shape:', X_scaled.shape, 'Classes:', set(y))
```

Part 2. Apply Classification Algorithms

Implement the following five classification algorithms covered in class using scikit-learn.

Algorithm Overview

Algorithm	scikit-learn class	Characteristics
Linear Classifier	sklearn.linear_model.Perceptron	Linear decision boundary; fast; simple baseline
Logistic Regression	sklearn.linear_model.LogisticRegression	Probabilistic linear classifier; highly interpretable
K-Nearest Neighbors	sklearn.neighbors.KNeighborsClassifier	Non-parametric, distance-based; sensitive to feature scaling
Gaussian Naive Bayes	sklearn.naive_bayes.GaussianNB	Assumes feature independence; fast; works on small data
Neural Network	sklearn.neural_network.MLPClassifier	Captures non-linear patterns; sensitive to hyperparameters

Reference Code: Defining the Models

```
from sklearn.linear_model import Perceptron, LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neural_network import MLPClassifier

models = {
    'Linear Classifier' : Perceptron(max_iter=1000, random_state=42),
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'KNN'               : KNeighborsClassifier(n_neighbors=5),
    'Gaussian NB'       : GaussianNB(),
    'Neural Network'    : MLPClassifier(hidden_layer_sizes=(64,),
                                         max_iter=500, random_state=42),
}
```

Evaluation Protocol

Running 10-fold cross validation on a model produces 10 accuracy values, from which you can compute the mean and standard deviation. We treat this as one experiment.

To stabilize the results, repeat this experiment 100 times. Across the resulting 1,000 accuracy values, compute the final mean and standard deviation.

You can do this in one shot using `RepeatedKFold`:

```
from sklearn.model_selection import RepeatedKFold, cross_val_score
```

```
# 10-fold CV repeated 100 times (1,000 train/eval rounds in total)
rkf = RepeatedKFold(n_splits=10, n_repeats=100, random_state=42)

results = {}
for name, model in models.items():
    scores = cross_val_score(model, X_scaled, y, cv=rkf,
                             scoring='accuracy', n_jobs=-1)
    results[name] = (scores.mean(), scores.std())
    print(f'{name:20s} mean={scores.mean():.4f} std={scores.std():.4f}')
```

Swap `model` for each of the five classifiers in turn. **Tip:** using `n_jobs=-1` lets all CPU cores run in parallel, so even 1,000 evaluations finish in a reasonable time.

Example: What Mean $\pm$ Std Tells You

If KNN reports mean=0.91 and std=0.03, it means that across 1,000 evaluations the accuracy was around 91% on average and varied by about $\pm 3\%$ from fold to fold. When two models have similar means, the one with the smaller std is generally the more stable model.

Reporting

Summarize the performance of all five algorithms in a table like the one below. Fill in the values from your own experiments.

Algorithm	Mean Accuracy	Std
Linear Classifier		
Logistic Regression		
KNN	0.91	0.03
Gaussian NB		
Neural Network		

What Your Part 2 Discussion Should Cover

- Which algorithm performed best, and a hypothesis about why.
- Whether the gaps between algorithms are meaningful relative to their std.
- How the dataset characteristics (sample size, number of features, linear vs. non-linear structure) likely shaped the results.

Part 3. Optimization via Feature Selection

When the dataset has m features, using all of them is not always optimal. Restricting the model to a subset of features can either improve or hurt performance, depending on the algorithm.

Goal

For each of the five algorithms, find **the feature subset that maximizes accuracy**.

Approach

In principle, the most accurate approach is exhaustive search — trying all $2^m - 1$ possible feature subsets. However, the number of subsets explodes with m , so this is rarely practical.

A Quick Intuition

With $m=10$ there are 1,023 subsets. With $m=20$ the number jumps to about one million, and with $m=30$ it reaches roughly one billion. If each subset requires 1,000 cross-validation runs, $m=20$ alone is essentially unfinishable.

Choose one of the following approaches:

- **Exhaustive search:** recommended only when m is small enough (e.g., $m \leq 15$) to enumerate all subsets.
- **Heuristic search:** apply a search algorithm covered in class (forward selection, backward elimination, or any algorithms you want). This is the practical choice when there are many features.

State which approach you chose and why in your report.

A Note on Forward / Backward Selection

Forward selection starts from the empty set and adds, at each step, the single feature that improves accuracy the most. Backward elimination starts with all features and removes, at each step, the feature whose removal hurts accuracy the least (or improves it).

Both are greedy algorithms: they commit to the locally best decision at every step and never reconsider it. This is much cheaper than exhaustive search (about $O(m^2)$ evaluations) but can miss the truly optimal subset, much like a hiker who always steps uphill — they might end up on a small hill rather than the tallest peak.

Aspect	Forward Selection	Backward Elimination
Start state	Empty set (0 features)	Full set (all features)
Direction	Add one feature at a time	Remove one feature at a time
Best when	Many features (cheap to start)	Features are strongly correlated

Aspect	Forward Selection	Backward Elimination
Weakness	Once a feature is skipped, its synergy with others may be missed	Initial training with all features can be expensive

Reference Code: Exhaustive Search

```

from itertools import combinations
import numpy as np

def exhaustive_search(model, X, y, feature_names, cv):
    best_acc, best_subset = -1, None
    m = X.shape[1]
    for k in range(1, m + 1):
        for combo in combinations(range(m), k):
            scores = cross_val_score(model, X[:, list(combo)], y,
                                     cv=cv, scoring='accuracy', n_jobs=-1)
            if scores.mean() > best_acc:
                best_acc = scores.mean()
                best_subset = [feature_names[i] for i in combo]
    return best_subset, best_acc

# Note: this becomes infeasible quickly when m grows (m > 15).

```

Reference Code: Forward / Backward Selection

```

from sklearn.feature_selection import SequentialFeatureSelector

# Forward selection example (Logistic Regression as the wrapper)
sfs = SequentialFeatureSelector(
    estimator=LogisticRegression(max_iter=1000),
    n_features_to_select='auto',    # or an integer
    direction='forward',           # 'backward' is also valid
    scoring='accuracy',
    cv=RepeatedKFold(n_splits=10, n_repeats=10, random_state=42),
    n_jobs=-1,
)
sfs.fit(X_scaled, y)
selected = np.array(feature_names)[sfs.get_support()]
print('Selected features:', list(selected))

```

Experimental Notes

- Use the same RepeatedKFold protocol as in Part 2 to keep the comparison fair. If runtime becomes an issue, you may reduce `n_repeats` — but state this clearly in your report.

- Apply the same scaling and encoding as in Part 2; only the choice of feature subset should change.
- When picking a "best" subset, consider both mean accuracy and std rather than the mean alone.

Reporting

For each algorithm, report the optimal feature subset found and the corresponding performance.

Algorithm	Best Feature Subset	Mean Accuracy	Std
Linear Classifier			
Logistic Regression			
KNN			
Gaussian NB			
Neural Network			

Then compare these numbers with the Part 2 results (which used all features) and briefly discuss the effect of feature selection. For example: "For Gaussian NB, accuracy improved as features were removed, presumably because variables that violated the independence assumption were dropped."

Submission

Do not submit source code files directly. Submit only a link to your GitHub repository.

Requirements

1. Create a personal GitHub repository.
2. Place your analysis code (Jupyter notebook or Python script) in the repository.
3. Write your report as README.md so that it renders directly on GitHub.
4. Keep the repository private until the deadline. Switch it to public immediately after the deadline.
5. Submit the GitHub repository URL on UTRGV Brightspace. (Specify the actual submission method.)

How to Change Repository Visibility

Go to GitHub Settings → General → Danger Zone → **Change visibility** and switch the repository to Public.

Suggested README.md Structure

```
# CSCI 3329 – Homework 3 Report

## 1. Dataset
- Name / source / number of samples / number of classes
- Class distribution (table or bar chart)

## 2. Preprocessing
- Missing-value handling
- Encoding and scaling decisions, with rationale

## 3. Part 2 – Algorithm Comparison
| Algorithm | Mean Accuracy | Std |
|-----|-----|-----|
| ...      | ...          | ... |

## 4. Part 3 – Feature Selection
- Search method and justification
| Algorithm | Best Feature Subset | Mean Accuracy | Std |
|-----|-----|-----|-----|

## 5. Discussion
- Part 2 vs Part 3 comparison
- Per-algorithm observations
- Limitations and ideas for improvement
```

6. Reproduction

- Python version, key library versions
- Run command (e.g., `python main.py`)

Pre-submission Checklist

- Does README.md render cleanly on the GitHub page?
- Are both the Part 2 and Part 3 result tables filled in?
- Can someone else reproduce your experiments? (random_state, requirements.txt, etc.)
- Is your name registered on the UCI dataset Google Sheet?
- Did you switch the repository to Public right after the deadline?
- Did you submit the correct GitHub URL on Brightspace?

Grading Rubric

The assignment is graded out of 100 points. Points reward not only correct execution but also depth of analysis and clarity of the report.

Item	Points	Evaluation Criteria
Dataset selection & introduction	10	Source, characteristics, and class distribution explained. UCI registration confirmed.
Preprocessing (Part 1)	15	Appropriate handling of missing values, encoding, and scaling; clean code.
Classifier implementation (Part 2)	25	All five algorithms run correctly; RepeatedKFold (10 x 100) applied.
Feature selection (Part 3)	25	Search method justified; correct implementation; per-algorithm best subsets reported.
Analysis & discussion	15	Depth of comparison between Part 2 and Part 3; meaningful insights.
Report (README.md)	10	Structure, readability, tables/charts, reproducibility info (seed, versions).

Common Reasons for Lost Points

- Forgetting to scale features, which causes KNN/MLP to perform unrealistically poorly.
- Using a single train/test split instead of RepeatedKFold.
- Not justifying the chosen search method in Part 3.
- Reporting tables without any comparison, interpretation, or discussion.
- Leaving the repository private after the deadline, making grading impossible.

Deadline

Due: TBA

Note: Late submissions follow the late policy in the course syllabus. The GitHub commit timestamp is used for grading, so commit incrementally as you work rather than pushing everything at the last minute.

References

- scikit-learn — Cross validation
- scikit-learn — Feature selection
- UCI Machine Learning Repository

- [GitHub Docs — Setting repository visibility](#)